Manejo de texto y expresiones regulares en SQL Server

Belerofontech

CONTENIDO

- ¿Por qué el texto da problemas?
- Funciones de texto esenciales
- Funciones de texto modernas (2017 → 2025)
- LIKE y PATINDEX: patrones sin regex
- Expresiones regulares: lo fundamental
- Funciones REGEXP_* en SQL Server 2025
- Casos prácticos, rendimiento y buenas prácticas

Por qué el texto da problemas

- **Datos de orígenes diversos:**
  - ERPs, Excel, formularios web, ficheros planos
- **Espacios invisibles:**
  - mayúsculas/minúsculas, tildes, separadores mezclados
- **Códigos "inteligentes"** con información embebida:
  - Ejemplo: FAC-2026-000123
- **Campos libres:**
  - teléfonos, emails, NIF, direcciones
- **El texto no correctamente tratado rompe joins, agrupaciones y KPI’s**

Funciones de texto esenciales

Longitud y tamaño: **LEN** vs **DATALENGTH**

- **LEN(s)**:
  - número de caracteres, **ignora espacios finales**
- **DATALENGTH(s)**:
  - número de **bytes** (cuenta espacios finales;
  - NVARCHAR = 2 bytes/carácter en UCS-2)
- Útil para detectar espacios ocultos:
  - WHERE LEN(Columna) <> DATALENGTH(Columna);

Extraer: **LEFT, RIGHT, SUBSTRING**

- **LEFT(s, n),**
- **RIGHT(s, n),**
- **SUBSTRING**(s, inicio, longitud), base 1
- *(2025)* **SUBSTRING**(s, inicio) con longitud opcional

Buscar: **CHARINDEX** y **PATINDEX**

- **CHARINDEX**(buscado, s [, inicio]):
  - posición del literal (0 si no está)
- **PATINDEX**('%patrón%', s):
  - posición del primer patrón tipo LIKE
- Combinados con **SUBSTRING** para trocear

Transformar: **REPLACE, STUFF, TRANSLATE, REPLICATE, REVERSE**

- **REPLACE**(s, a, b): todas las apariciones de un literal
- **STUFF**(s, inicio, n, nuevo): borra n caracteres e inserta
- *(2017+)* **TRANSLATE**(s, 'abc', 'xyz'): sustituye carácter a carácter
- **REPLICATE**(c, n): relleno con ceros, máscaras
- **REVERSE**(s): truco para buscar "la última aparición"

Limpiar: **TRIM, LTRIM, RTRIM, UPPER, LOWER**

- *(2017+)* **TRIM**(s) y **TRIM**('chars' FROM s)
- *(2022+)* **TRIM**(LEADING | TRAILING | BOTH 'chars' FROM s), **LTRIM**(s, 'chars'), **RTRIM**(s, 'chars')
- **UPPER**, **LOWER** para normalizar mayúsculas y minúsculas
- Ojo:
  - **CHAR**(160), tabuladores **CHAR**(9), saltos **CHAR**(13)+**CHAR**(10) **no** son espacios para **TRIM** por defecto

Concatenar: **+, CONCAT, CONCAT_WS, ||**

- **+** con un NULL → resultado NULL
- **CONCAT**(…): trata NULL como cadena vacía
- *(2017+)* **CONCAT_WS**(sep, …): separador y salta NULL
- *(2025)* operador **||** estándar ANSI (y **||=**)

Agregar y partir: **STRING_AGG y STRING_SPLIT**

- *(2017+)* **STRING_AGG**(col, sep) WITHIN GROUP (ORDER BY …)
- *(2016+)* **STRING_SPLIT**(s, sep); *(2022+)* tercer parámetro enable_ordinal → columna ordinal
- **STRING_SPLIT** solo admite separador de **un carácter** (para separadores complejos → REGEXP_SPLIT_TO_TABLE)

LIKE y PATINDEX

LIKE: los comodines

Comodín	Significado	Ejemplo
%	0 o más caracteres	'FAC%'
_	exactamente 1 carácter	'C__1'
[abc]	un carácter del conjunto	'[AEIOU]%'
[a-f]	un carácter del rango	'[0-9][0-9]%'
[^abc]	un carácter que **no** esté	'%[^0-9]%'

LIKE: escapar comodines y trampas habituales

- Buscar un % o _ literal:
  - ESCAPE o corchetes
- Corchete literal: [[]
- LIKE respeta la collation (mayúsculas/acentos)
- Espacios finales:
  - en el patrón importan, en la columna CHAR hay relleno
- Unicode:
  - usar N'...' con columnas NVARCHAR

LIKE y rendimiento (sargabilidad)

- LIKE 'ABC%' → puede usar **index seek**
- LIKE '%ABC' o '%ABC%' → **scan** completo
- Funciones sobre la columna (UPPER(col) LIKE …) → scan
- Alternativas: columna calculada persistida + índice, full-text search, columna normalizada

Límites de LIKE / PATINDEX: ¿regex?

- No hay cuantificadores: "uno o más dígitos" no se puede expresar
- No hay alternativas (A|B) ni grupos
- No hay anclajes reales dentro del patrón
- No se puede **extraer** ni **reemplazar** por patrón
- Validar un email o un NIF con LIKE = patrones larguísimos o imposibles

Expresiones regulares REGEX: Fundamentales

Regex 1: qué es y piezas básicas

- Una expresión regular es un **patrón** que describe un conjunto de cadenas
- Sirve para: **validar**, **buscar**, **extraer**, **reemplazar**, **partir**
- Literales: abc coincide con "abc"
- Metacaracteres con significado especial: . ^ $ * + ? ( ) [ ] { } | \
- Para usarlos como literal, se escapan: \. \(\$
- . = cualquier carácter (excepto salto de línea, salvo flag s)

Regex 2: clases de caracteres

Patrón	Significado
[abc]	a, b o c
[a-z0-9]	rango
[^0-9]	cualquiera excepto dígitos
\d / \D	dígito / no dígito
\w / \W	carácter de palabra [A-Za-z0-9_] / lo contrario
\s / \S	espacio en blanco (espacio, tab, salto) / lo contrario
[[:alpha:]], [[:digit:]], \p{L}	clases POSIX / Unicode

Regex 3: cuantificadores

Cuantificador	Significado
*	0 o más
+	1 o más
?	0 o 1 (opcional)
{n}	exactamente n
{n,}	n o más
{n,m}	entre n y m
*? +? ??	versiones perezosas (lo mínimo posible)

Regex 4: anclajes y alternativas

- ^ inicio de cadena · $ fin de cadena
- Con flag m (multilínea): ^ y $ actúan por línea
- \b límite de palabra
- | alternativa: FAC|ABO|PRO
- **Validar = anclar**: ^patrón$

Regex 5: grupos, capturas y referencias

- (…) agrupa y **captura**
- (?: …) agrupa **sin** capturar
- Los grupos se numeran por su paréntesis de apertura: \1, \2…
- Se usan para **extraer partes** y para **reordenar en un reemplazo**
- Grupos con nombre: (?P<anio>\d{4})
- Motor de SQL Server = **RE2**: no soporta *lookahead/lookbehind* ((?=…), (?<=…)) ni retroreferencias dentro del patrón; a cambio, tiempo de ejecución lineal y sin "backtracking catastrófico"

Funciones REGEXP_* en SQL Server 2025

Función	Devuelve	Para qué
REGEXP_LIKE	booleano	Filtrar / validar (WHERE, CHECK)
REGEXP_COUNT	int	Contar coincidencias
REGEXP_INSTR	int	Posición de una coincidencia
REGEXP_SUBSTR	texto	Extraer una coincidencia (o un grupo)
REGEXP_REPLACE	texto	Reemplazar por patrón
REGEXP_MATCHES	tabla	Todas las coincidencias con sus grupos
REGEXP_SPLIT_TO_TABLE	tabla	Partir por patrón

Flag	Efecto
c	sensible a mayúsculas (por defecto)
i	insensible a mayúsculas
m	multilínea: ^ y $ por línea
s	. también coincide con salto de línea

- **Disponible en SQL Server 2025**
- REGEXP_LIKE, REGEXP_MATCHES y REGEXP_SPLIT_TO_TABLE requieren **nivel de compatibilidad 170**
  - ALTER DATABASE NameDatabase SET COMPATIBILITY_LEVEL = 170;
- Flags
  - Se combinan: 'im'
  - **No** dependen de la collation de la columna

Rendimiento: cuándo sí y cuándo no

- Las funciones REGEXP_* sobre una columna **no usan índices** (no son sargables)
- Si existe un prefijo literal, combinarlo con LIKE para reducir filas primero
- Para validaciones recurrentes: columna calculada persistida o CHECK
- Para búsqueda de palabras en textos grandes: **Full-Text Search**
- LIKE/CHARINDEX siguen siendo más baratos cuando bastan
- Revisar las limitaciones documentadas para tipos LOB (VARCHAR(MAX)/NVARCHAR(MAX)) y tamaño de patrón

Buenas prácticas

- Anclar con ^...$ cuando se valida
- Preferir clases explícitas a .*
- Cuidado con \w y el español: usar \p{L}
- Comentar los patrones complejos y guardarlos con ejemplos de prueba
- Probar los patrones fuera (regex101 con sabor **Golang/RE2**) antes de llevarlos a SQL
- Usar flags en vez de UPPER() para insensibilidad a mayúsculas
- No usar regex para lo que hace REPLACE o LEFT
- Validar formato con regex, **lógica** con SQL

Elecciones clave:

Necesito...	Antes de 2025	SQL Server 2025
Filtrar por patrón sencillo	LIKE	LIKE
Validar formato complejo	CLR / imposible	REGEXP_LIKE
Posición de un patrón	PATINDEX	REGEXP_INSTR
Extraer una parte variable	SUBSTRING + CHARINDEX	REGEXP_SUBSTR
Reemplazar literal	REPLACE	REPLACE
Reemplazar por patrón	bucles / CLR	REGEXP_REPLACE
Partir por 1 carácter	STRING_SPLIT	STRING_SPLIT
Partir por patrón	CLR	REGEXP_SPLIT_TO_TABLE
Contar apariciones	LEN – LEN(REPLACE)	REGEXP_COUNT